

InstanciaIris: Instanciação, Execução e Validação de um Projeto de Machine Learning a partir do Template da FábricaIA

Diego Cordeiro de Oliveira¹

¹ Programa de Pós-Graduação em Ciência da Computação (PPGCC)
Departamento de Computação – Centro de Ciências da Natureza
Universidade Federal do Piauí (UFPI) – Teresina – PI – Brazil

diego.cordeiro@ufpi.edu.br

Abstract. *This technical report presents the instantiation and end-to-end execution of the InstanciaIris project, a machine learning workflow built upon the FabricaIA template. The objective is to execute the pipeline for the Iris dataset, covering preprocessing (label encoding and feature standardization), training of a Random Forest classifier with experiment tracking using MLflow, model persistence, and exposure of a REST prediction endpoint via FastAPI. The process was executed without errors and was recorded in the instanciairis_experiments experiment in MLflow. The prediction API successfully responded to a test request with the predicted class and its associated probabilities.*

Resumo. *Este relatório técnico apresenta a instanciação e a execução ponta a ponta do projeto InstanciaIris, um fluxo de aprendizado de máquina construído sobre o template FabricaIA. O objetivo executar o pipeline para o dataset Iris, abrangendo pré-processamento (codificação do rótulo e padronização das características), treinamento de um classificador Random Forest com rastreamento de experimentos em MLflow, persistência do modelo e exposição de um endpoint de predição REST via FastAPI. O processo foi executado sem erros, sendo registrado no experimento instanciairis_experiments do MLflow. A API de predição respondeu a uma requisição de teste com a classe e as probabilidades associadas.*

1. Introdução

Sistemas baseados em aprendizado de máquina não se encerram no treinamento de um modelo. A reprodutibilidade, o rastreamento de experimentos, a disponibilização do modelo para inferência e a implantação da solução são etapas igualmente relevantes para consolidar uma solução confiável. Nesse contexto, o uso de templates e de ferramentas de experimentação contribui para padronizar o fluxo de trabalho e reduzir o risco de perda de informação sobre versões, hiperparâmetros e métricas.

O dataset Iris, proposto por [Fisher 1936], é um benchmark clássico de classificação supervisionada, com 150 amostras e três classes de espécies de flor, sendo adequado para validar pipelines de pré-processamento, treinamento e avaliação.

Diante disso, este relatório documenta a instanciação e a execução do projeto InstanciaIris, com os seguintes objetivos: (i) instanciar um projeto de aprendizado de máquina a partir do template FabricaIA; (ii) construir um pipeline de pré-processamento e

treinamento com rastreamento de experimentos em MLflow; (iii) expor um modelo treinado por meio de uma API REST com FastAPI; e (iv) validar o funcionamento ponta a ponta por meio de testes automatizados e de uma chamada de predição real.

2. Repositório do Projeto

O projeto *InstanciaIris* foi instanciado a partir do template disponibilizado pela FábricaIA e está disponível no repositório GitHub:

<https://github.com/diegocordeiro/InstanciaIris.git>

3. Fundamentação Teórica

O aprendizado de máquina supervisionado consiste em uma abordagem na qual um modelo é treinado a partir de um conjunto de exemplos previamente rotulados, com o objetivo de aprender uma função capaz de estabelecer uma relação entre as características de entrada e o respectivo rótulo de saída. Após o treinamento, o modelo pode utilizar essa relação para realizar previsões sobre novos dados não observados.

O Random Forest é um método de conjunto (ensemble) baseado em árvores de decisão, no qual múltiplas árvores são construídas sobre subamostras aleatórias dos dados e de suas características, e a predição final é obtida por votação majoritária [Breiman 2001, Dietterich 2000]. A implementação utilizada neste trabalho é a fornecida pela biblioteca scikit-learn [Pedregosa et al. 2011].

O dataset Iris [Fisher 1936] contém 150 amostras organizadas em três classes (setosa, versicolor e virginica), caracterizadas por quatro medidas morfológicas: comprimento / largura da sépala e comprimento / largura da pétala.

Em tarefas de classificação, o pré-processamento é tipicamente composto pela codificação de variáveis categóricas e pela padronização de características numéricas. A codificação de rótulos (LabelEncoder) transforma as classes em valores inteiros, enquanto a padronização (StandardScaler) [Pedregosa et al. 2011] ajusta cada característica para média zero e desvio padrão unitário, evitando que a diferença de escala influencie o modelo.

O rastreamento de experimentos é fundamental para a reprodutibilidade em aprendizado de máquina. O MLflow [Zaharia et al. 2018] permite registrar experimentos, parâmetros, métricas e artefatos de modelos, organizados em execuções (runs) agrupadas em experimentos. Neste trabalho, o MLflow foi configurado com backend SQLite local (sqlite:///mlflow.db).

Para a avaliação de modelos de classificação, métricas como acurácia, precisão, recall e F1-score, derivadas da matriz de confusão, são amplamente utilizadas [Sokolova and Lapalme 2009]. A divisão dos dados em conjuntos de treino e teste é uma prática padrão de validação, e a estratificação é comumente empregada para preservar a distribuição das classes [Kohavi 1995]. Na busca por melhor desempenho, a otimização de hiperparâmetros, por busca aleatória ou em grade, é frequentemente utilizada [Bergstra and Bengio 2012].

Para disponibilizar o modelo, utiliza-se uma API REST construída com FastAPI [Ramirez], framework web assíncrono que emprega modelos de dados definidos com Pydantic para validar as requisições disponível no template.

4. Metodologia

4.1. Ambiente e estrutura do projeto

O projeto foi instanciado em um ambiente virtual Python 3.12.9, com as bibliotecas scikit-learn 1.9.0, MLflow 3.16.0, pandas, numpy e FastAPI. A estrutura de diretórios segue o template FabricaIA, com as pastas config/, data/, models/, notebooks/, src/, tests/ e docs/.

4.2. Dados e pré-processamento

O arquivo data/raw/iris.csv contém 150 linhas e 5 colunas (quatro características numéricas e a coluna alvo target). O pipeline de pré-processamento envolve limpeza, codificação do rótulo e padronização das características, seguido da divisão dos dados em treino e teste (80/20), resultando em 119 amostras de treino.

```
df = load_data(project_path("data", "raw", "iris.csv"))
df_clean = clean_data(df, drop_duplicates=True)
df_encoded = encode_categorical(df_clean) # alvo -> 0/1/2
df_scaled = scale_features(df_encoded) # 4 features (StandardScaler)
X_train, X_test, y_train, y_test = split_data(df_scaled, "target")
```

4.3. Treinamento e rastreamento de experimentos

O classificador Random Forest foi configurado com 100 estimadores e profundidade máxima 10, treinado por meio do componente ModelTrainer do projeto. O rastreamento foi realizado com MLflow, no experimento instanciairis_experiments, com backend SQLite local, registrando parâmetros (n_estimadores, max_depth e número de amostras), métricas e o modelo como artefato.

```
model = trainer.train_model(X_train, y_train, "random_forest",
                             run_name="random_forest_baseline", n_estimators=100, max_depth
                             =10)
metrics = trainer.evaluate_model(model, X_test, y_test)
trainer.save_model(model, project_path("models", "trained", "model.
    pkl"),
                    artifact_path="random_forest_model")
trainer.end_run()
```

Em uma segunda execução, a última célula do notebook treinou novamente o classificador Random Forest sem fixar os hiperparâmetros (valores padrão), gerando um novo run no MLflow sob o nome entrega_topicos_avancados. Esse run alcançou acurácia de 0,9667, precisão 1,00, recall 0,90 e F1 0,9474.

```
trainer = ModelTrainer(
    model_type="classification",
    config_path=str(project_path("config", "config.yaml")))
model = trainer.train_model(X_train, y_train, 'random_forest',
                             run_name='entrega_topicos_avancados')
metrics = trainer.evaluate_model(model, X_test, y_test)
trainer.end_run()
```

4.4. API de predição

Na inicialização, a aplicação FastAPI carrega os modelos de `models/trained/*.pkl` e deriva o nome do modelo removendo o sufixo `_model`. O endpoint `POST /predict` recebe um corpo JSON com as características e o nome do modelo e retorna a classe predita, as probabilidades e a confiança.

```
POST /predict
{ "features": {
  "sepal_length": 5.1, "sepal_width": 3.5,
  "petal_length": 1.4, "petal_width": 0.2
},
  "model_name": "model" }
```

A resposta retornada é exibida abaixo.



Figura 1. Resposta da API de predição.

4.5. Validação por testes automatizados

A suíte de testes do projeto foi executada e validou os componentes de processamento, treinamento, engenharia de características e visualização, com 18 testes aprovados.



Figura 2. Relatório teste automatizados.

5. Resultados e Discussão

Os resultados obtidos confirmam a execução do pipeline. A Tabela 1 resume as métricas do modelo Random Forest treinado em dois momentos e registradas no experimento `instanciairis_experiments` com MLflow. A figura 3 por sua vez, apresenta o registro do treinamento no MLflow, incluindo os hiperparâmetros utilizados pelo modelo Random Forest, bem como as métricas de avaliação e os artefatos gerados durante a execução.

Tabela 1. Métricas registradas no MLflow (precisão, recall e F1 para a classe 1).

Run	Acurácia	Precisão	Recall	F1
random_forest_baseline	0,9333	0,90	0,90	0,90
entrega_topicos_avancados	0,9667	1,00	0,90	0,9474

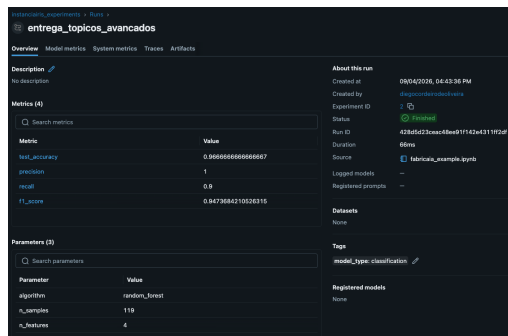


Figura 3. Registro do treinamento MLFlow.

5.1. Análise exploratória e importância das características

A Figura 4 apresenta a distribuição de duas das quatro características do dataset Iris (sepal_length e sepal_width), selecionadas para a análise exploratória, permitindo observar o comportamento dos respectivos valores na amostra analisada. A Figura 5, por sua vez, apresenta a matriz de correlação entre as características do dataset, possibilitando avaliar a existência de possíveis relações lineares entre as variáveis. Por fim, a Figura 6 apresenta a importância relativa atribuída pelo modelo *Random Forest* a cada característica, indicando sua contribuição para as decisões realizadas pelo modelo.

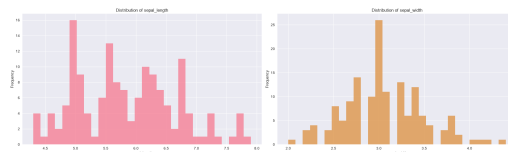


Figura 4. Distribuição de duas características do dataset Iris (sepal_length e sepal_width).

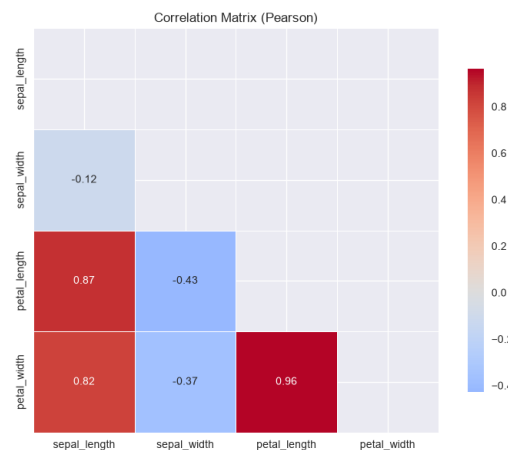


Figura 5. Matriz de correlação entre as características do dataset Iris.

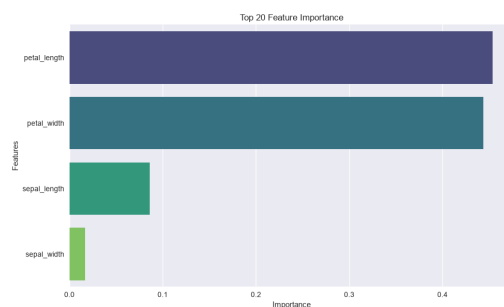


Figura 6. Importância das características atribuída pelo modelo Random Forest.

O modelo alcançou acurácia de 93,33% e F1 de 0,90, resultado coerente com a simplicidade e a separabilidade do dataset Iris. Como esperado para um modelo baseline, não foi realizada otimização de hiperparâmetros, a escolha de 100 estimadores e profundidade máxima 10 forneceu um desempenho estável. Já o run entrega_topicos_avancados, treinado com os hiperparâmetros padrão do Random Forest, alcançou métricas superiores (acurácia de 0,9667, precisão de 1,00, recall de 0,90 e F1 de 0,9474), evidenciando o efeito da configuração de hiperparâmetros. O modelo foi persistido em models/trained/model.pkl e registrado como artefato no experimento instanciairis_experiments.

6. Conclusão

Este relatório documentou o processo de instanciação, desenvolvimento e execução do projeto InstanciaIris, a partir do template disponibilizado pela FabricaIA. Os objetivos propostos foram alcançados, contemplando a construção e a execução do pipeline de pré-processamento e treinamento utilizando Jupyter Notebook. O modelo Random Forest foi treinado, avaliado e devidamente rastreado por meio do MLflow.

Além disso, o modelo treinado foi persistido e disponibilizado por meio de uma API REST desenvolvida com FastAPI. Por fim, seu funcionamento foi validado por meio de testes automatizados e de uma chamada de predição realizada sobre dados reais, confirmando a integração entre as etapas de treinamento, rastreamento, persistência e disponibilização do modelo.

Referências

- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
- Dietterich, T. G. (2000). Ensemble methods in machine learning. In *Multiple Classifier Systems (MCS)*, pages 1–15. Springer.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.
- Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1137–1143.

- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.
- Ramirez, S. Fastapi documentation. <https://fastapi.tiangolo.com>. Acessado em 2026.
- Sokolova, M. and Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing and Management*, 45(4):427–437.
- Zaharia, M., Chen, A., Davidson, A., et al. (2018). Accelerating the machine learning lifecycle with mlflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.